© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_14

14. Seeing Is Believing: Multimodal LLMs and Image Hacking

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to understand the tools available in Java for image processing and the current capabilities generative AI (Gen AI) provides related to image and multimodal content processing.

Introduction

In an era where digital content streams in a rich tapestry of images, text, audio, and video, the ability to seamlessly process and understand multimodal data has become a defining feature of next-generation AI systems. Traditionally, image processing in Java has relied on mature libraries such as OpenCV—accessed via JNI through projects like OpenPnP OpenCV—and OCR engines like Tesseract to extract text from static media. While these tools excel at domain-specific tasks, they often require separate processing steps and bespoke integration layers to handle different data types.

The advent of multimodal large language models (LLMs) ushers in a transformative approach: one unified model that can jointly reason over images, text, and other modalities. By embedding dedicated encoders for each input type and consolidating reasoning within a shared transformer backbone, these models break down the silos between vision and language. As a result, developers can replace complex processing pipelines with concise prompts that yield structured, JSON-ready outputs, or even generate new media on demand.

This chapter explores how to bridge established Java tooling with the latest generative AI capabilities. We begin by surveying the core image-processing libraries available to Java developers. We then introduce the principles behind multimodal LLMs and demonstrate their power through a hands-on Spring Boot application that scans customer flight tickets and returns structured data. Building upon this foundation, we show how to en-

rich user experiences by generating destination-inspired images and outline extensions to other modalities, from audio sentiment analysis to synthesized speech. By the end of this chapter, you'll understand both the classical and cutting-edge techniques for multi-modal content processing in Java, and you'll be equipped to architect your own unified AI-driven workflows.

Tooling in Java for Image Processing

Image processing involves use of machine learning technologies and Optical Character Recognition (OCR) to process them. Machine learning generally involves the use of the Python language by using OpenCV library, which can be referenced at https://opencv.org. It supports the processing of images with support for real-time processing. OpenCV is written in C, there are relevant Java bindings for it through a project named OpenPnP OpenCV, which can be referenced at https://github.com/openpnp/opencv. It uses Java Native Interface (JNI) to load and operate OpenCV natively in Java. The API for classes and methods used in Java are similar to those in the OpenCV C project, encouraging the adoption of the Java library. This can be used by using the Maven-specific library package at https://mvnrepository.com/artifact/org.openpnp/opencv, as shown in Figure 14-1.

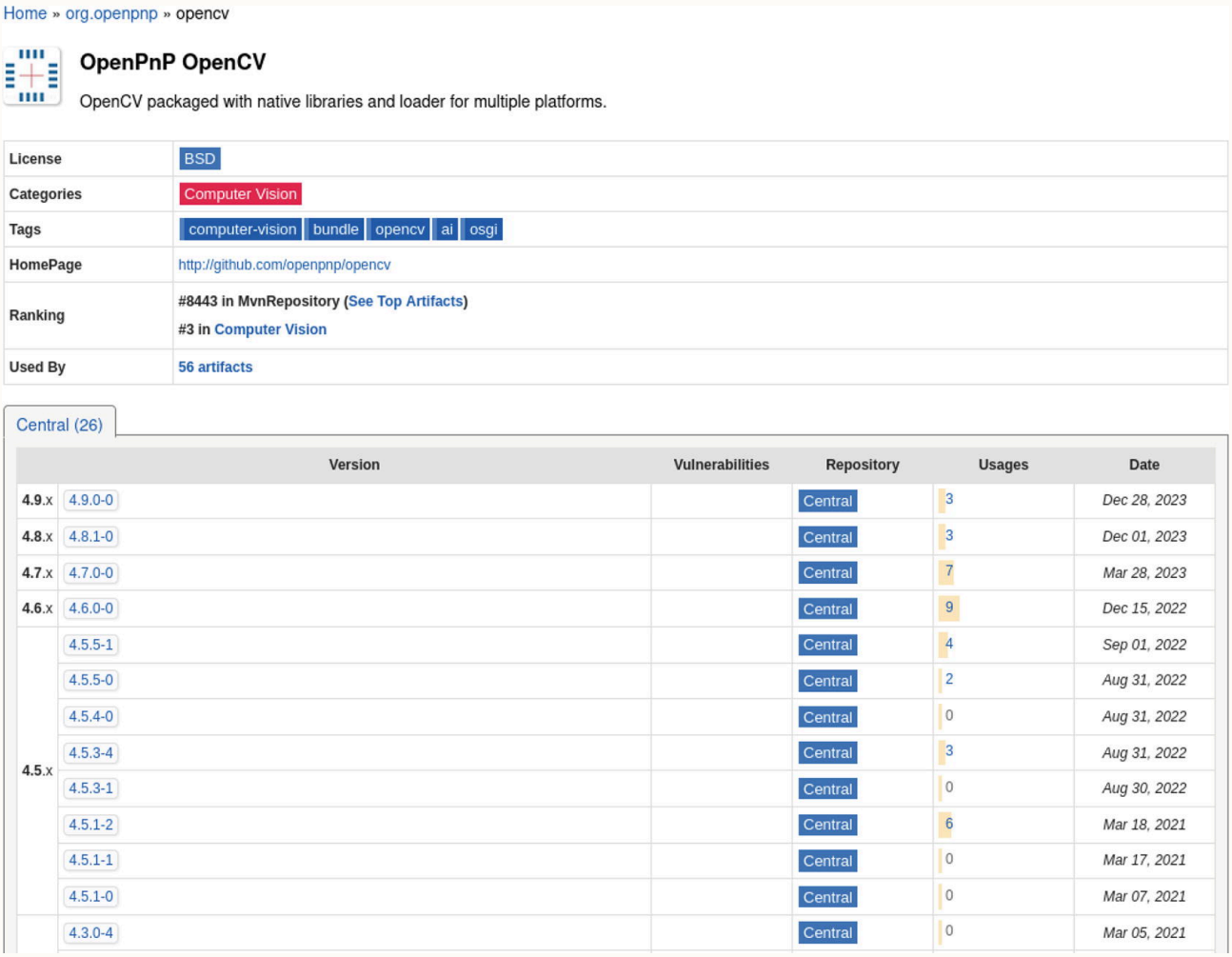


Figure 14-1 OpenPnP Maven repository

Similarly, the Tesseract project helps in recognizing characters. OCR helps scan and retrieve the text present in a variety of formats like PDF, images, etc. The tesseract-platform Maven package can help you use Tesseract in your Java projects, as shown in Figure 14-2.

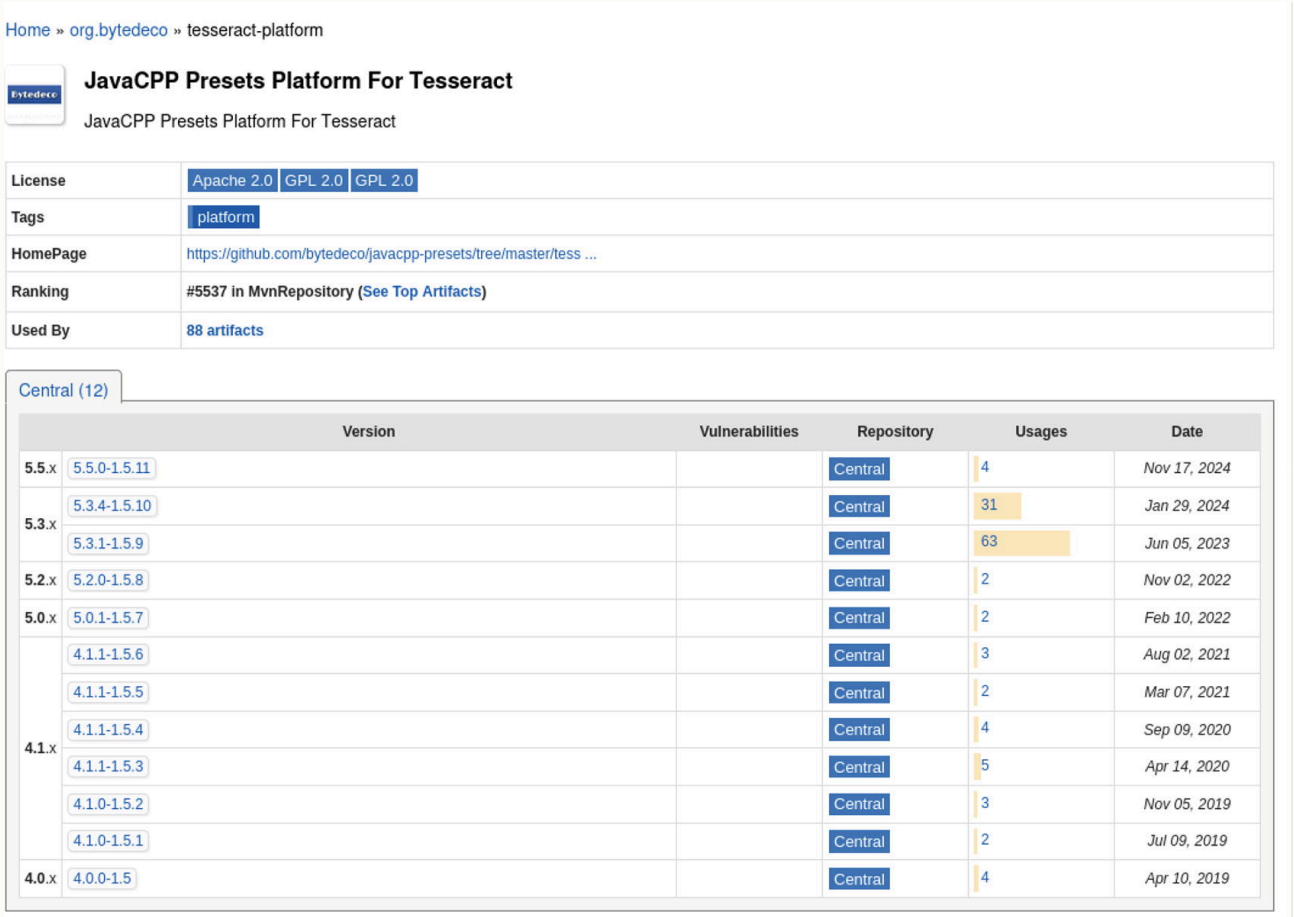


Figure 14-2 Maven repository for tesseract-platform

You can plug in these technologies and tools as different components and chain them together to process images, but it only takes information in a single format generally, and you may have to make additional requests if you need to further process the information. How about using a single tool to process the image and also do other processing on it too? Yes, multimodal LLMs help you to do this, discussed in the next section.

Multimodal LLMs

Multimodal LLMs are an extension of traditional text-only LLMs that can accept and jointly reason over multiple types of data—most commonly text and images but increasingly audio, video, or even structured sensor inputs. By integrating dedicated encoders for each modality and a shared transformer-based core, these models learn to correlate visual patterns with linguistic concepts, enabling tasks that neither pure vision nor pure language models could tackle alone. Early successes in this space include models that align images and captions (e.g., CLIP), but the latest generation—such as GPT-4 with vision, Google's PaLM-E, and Meta's Florence—goes further by allowing users to pose

free-form natural-language queries about images and receive coherent, context-aware responses. These models are bound to change with time and are to act as guidance to always verify the latest from the relevant vendor publishers and from Hugging Face.

The importance of multimodal LLMs lies in their ability to break down the barriers between how we see and how we speak. In practical terms, they power applications like visual question answering (e.g., "What ingredients are on this pizza?"), document understanding (e.g., extracting structured data from forms and invoices), and accessible interfaces for people with visual impairments. In robotics and autonomous systems, grounding language in real-world perception enables more intuitive control ("Pick up the red mug next to the keyboard"), while in creative domains, artists can sketch a scene and have the model generate evocative descriptions or even continue the artwork in code.

Beyond new use cases, multimodal LLMs also enhance robustness and disambiguation. Visual context can resolve linguistic ambiguities—distinguishing "bank" as a financial institution versus a riverbank when shown a relevant image—and language can guide vision modules toward task-specific observations. As these models continue to improve, they promise to form the backbone of "generalist" AI assistants, capable of seamlessly switching between reading, writing, and perceiving the world through multiple senses, thereby bringing us closer to more natural, human-like interactions with machines.

Building an Application to Process Flight Tickets

Returning to the airline company example, say you have a new requirement where you want to process flight tickets shared by your customers. The first task is to scan all customer relevant information from the ticket.

Let's start by creating a project from Spring Initializr with dependencies, as shown in Figure 14-3.

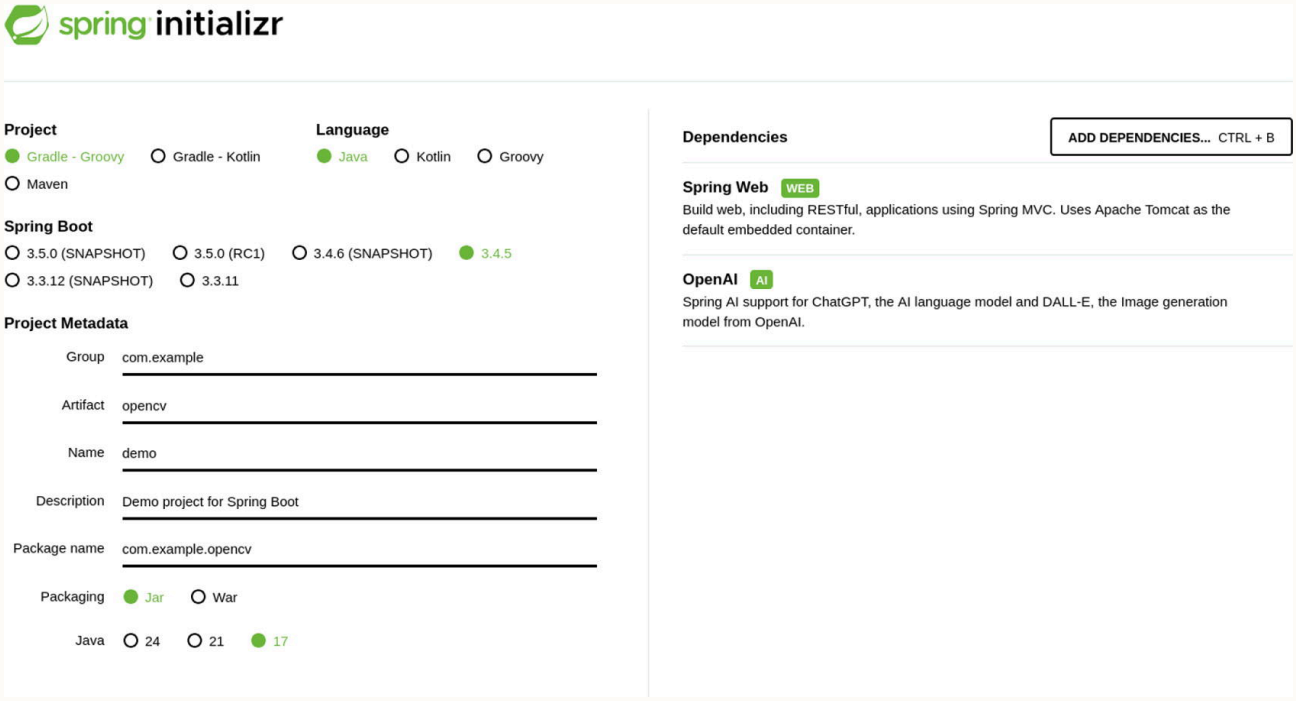


Figure 14-3 Initializing Spring Project with dependencies

Download the zip and unzip it at a location of your choice. The following is the final version of the build.gradle file:

File: ./build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.4.5'
    id 'io.spring.dependency-management' version '1.1.7'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}
repositories {
    mavenCentral()
}
ext {
    set('springAiVersion', "1.0.0-M8")
}
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.ai:spring-ai-starter-model-openai'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
```

```
}
dependencyManagement {
    imports {
        mavenBom "org.springframework.ai:spring-ai-bom:${springAiVersion}"
    }
}
tasks.named('test') {
    useJUnitPlatform()
}
```

Before you boot up your application, make sure to set your OpenAI API key in the application.properties file, as shown here:

File: ./src/main/resources/application.properties

```
spring.application.name=opencv
spring.ai.openai.api-key=YOUR_OPENAI_API_KEY
```

The goal is to scan and read the content in the sample ticket, as shown in Figure 14-4.



Figure 14-4 Sample flight ticket

This would generally be uploaded by the user through an API request, but to keep it simple here and limit the focus to the task at hand, you will store it in the resources folder with the name flight_ticket.jpg.

Now that the basic foundational setup is done, let's view the different components we will build to achieve the goal at hand. There are four components to it, as shown in Figure 14-5. The first component is the controller, which receives the POST API request. On receiving the request, it fetches the image from the resource directory and makes a call to the Service component. The Service component uses the injected OpenAI component to scan the contents of the input image.

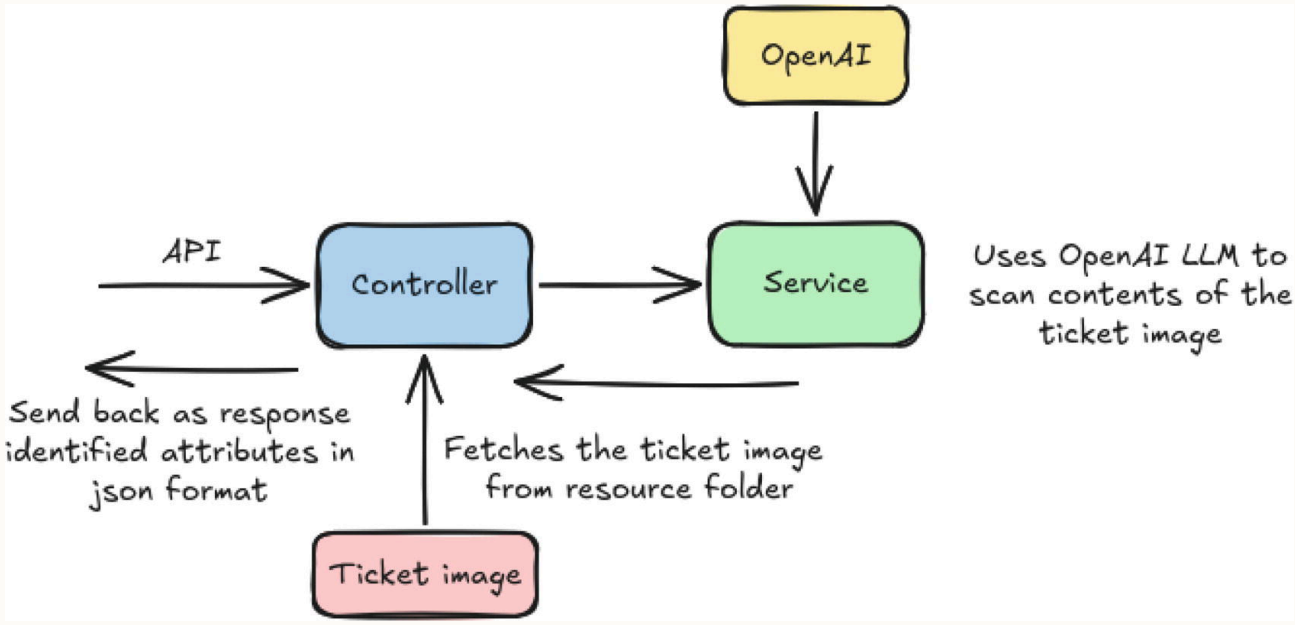


Figure 14-5 Components of the Spring Boot application

To represent the response flight info, we will create a DTO object. It will contain some basic getter and setter methods related to the information present in our flight ticket like name, flightNumber, origin, destination, date, time, seatNumber, etc. We will not go too deep into its explanation as it is a standard practice in Java enterprise applications, and the code is self-explanatory. The full code for this DTO object is as follows:

File: ./src/main/java/com/example/opencv/model/FlightInfo.java

```
package com.example.opencv.model;
import java.time.LocalDate;
import java.time.LocalTime;
public class FlightInfo {
    private String name;
    private String flightNumber;
    private String origin;
    private String destination;
    private LocalDate date;
    private LocalTime time;
    private String seatNumber;
// No-arg constructor
   public FlightInfo() { }
// All-args constructor
   public FlightInfo(String name,
                     String flightNumber,
                     String origin,
                     String destination,
                     LocalDate date,
                     LocalTime time,
                     String seatNumber) {
```

```
        this.name = name;
        this.flightNumber = flightNumber;
        this.origin = origin;
        this.destination = destination;
        this.date = date;
        this.time = time;
        this.seatNumber = seatNumber;
    }
// Getters and setters
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getFlightNumber() {
        return flightNumber;
    }
    public void setFlightNumber(String flightNumber) {
        this.flightNumber = flightNumber;
    }
    public String getOrigin() {
        return origin;
    }
    public void setOrigin(String origin) {
        this.origin = origin;
    }
    public String getDestination() {
        return destination;
    }
    public void setDestination(String destination) {
        this.destination = destination;
    }
    public LocalDate getDate() {
        return date;
    }
    public void setDate(LocalDate date) {
        this.date = date;
    }
    public LocalTime getTime() {
        return time;
    }
    public void setTime(LocalTime time) {
        this.time = time;
    }
    public String getSeatNumber() {
```

```
        return seatNumber;
    }
    public void setSeatNumber(String seatNumber) {
        this.seatNumber = seatNumber;
    }
}
```

Now, let's build our first component, which is the Service component. We will name it ScanService.java. The ScanService constructor will receive one injected argument, which is the OpenAI ChatModel instance. This is created by default by Spring AI as a bean when the application is booted.

Figure 14-6 represents the different components and classes.

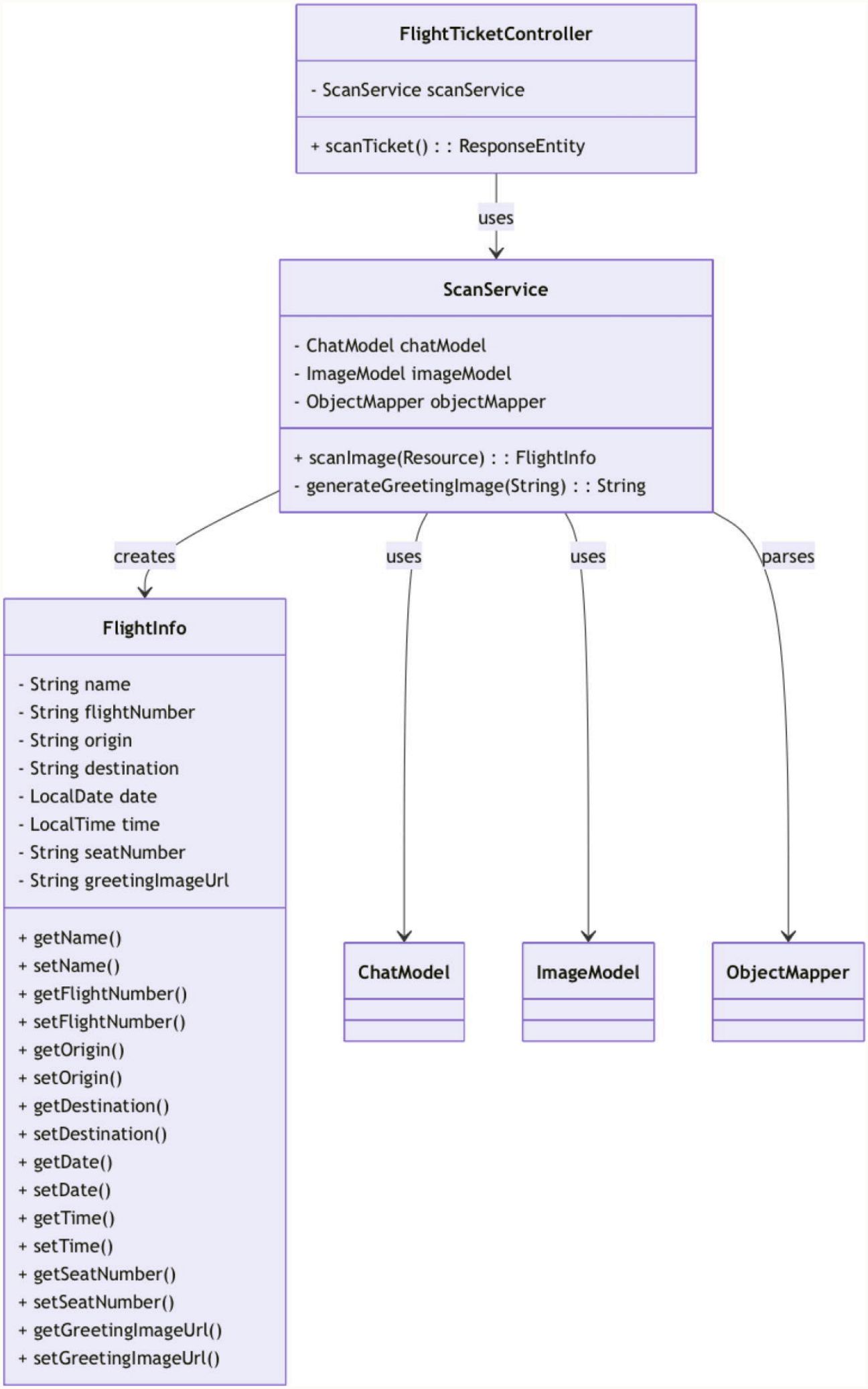


Figure 14-6 Components and classes

```
public ScanService(ChatModel chatModel) {
    this.chatModel = chatModel;
}
```

We will next define a method named scanImage that takes as an argument the image resource and returns the FlightInfo object.

```
public FlightInfo scanImage(Resource imageFile) {
```

There are three steps for setting up the request to OpenAI LLM to process the image:

1. Create an appropriate prompt requesting to process the image and fetch the relevant information.
2. Send the resource image in the request.
3. You do not just want the information in textual format but in a specific JSON format.

You will create a text prompt to request the LLM to scan the content of the flight ticket and also request to send it in an appropriate JSON format. Also, you can add an example JSON to give it a hint through one-shot prompting. The UserMessage API can be used for this purpose. It takes two arguments. The first is the text prompt, and the second is the additional file. The following code snippet shares this in detail. The word json in the prompt is important since it tells the LLM to produce the output in JSON format.

```
UserMessage userMessage = new UserMessage("""
      Scan the information in the Flight Ticket.
      Return in json format with keys like following example:
      {
         "name": "Jane Doe",
         "flightNumber": "XY1234",
         "origin": "SFO",
         "destination": "LHR",
         "date": "2025-06-01",
         "time": "14:45:00",
         "seatNumber": "21C"}
      """,
       new Media(MimeTypeUtils.IMAGE_JPEG, imageFile));
```

The user message object by itself doesn't do much and has to be used and passed to the OpenAI chat options request call. You will use the OpenAiChatOptions API to create the request object specifying the model to be used and what the response format should

be. You can create a ResponseFormat object by specifying the response to be the JSON_OBJECT type.

```
ResponseFormat responseFormat = ResponseFormat.builder()
        .type(ResponseFormat.Type.JSON_OBJECT).build();
OpenAiChatOptions openAiChatOptions = OpenAiChatOptions.builder()
        .model(OpenAiApi.ChatModel.GPT_4_O_MINI.getValue())
        .responseFormat(responseFormat)
        .build();
```

The final step is to call the OpenAI API by invoking the call method on the ChatModel API, as shown here:

```
ChatResponse response = chatModel.call(new Prompt(userMessage, openAiChatOptio
ns));
```

Since the response is in text format and our method returns the FlightInfo object as response, you will need to map the text json to the DTO object by using ObjectMapper.

```
this.objectMapper = new ObjectMapper()
        .registerModule(new JavaTimeModule())
        .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
String jsonResponse = Objects.requireNonNull(
        response.getResult().getOutput().getText()).trim();
return objectMapper.readValue(jsonResponse, FlightInfo.class);
```

The full source code for the ScanService.java file is shown here:

File: ./src/main/java/com/example/opencv/service/ScanService.java

```
package com.example.opencv.service;
import com.example.opencv.model.FlightInfo;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.model.ChatResponse;
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.content.Media;
import org.springframework.ai.openai.OpenAiChatOptions;
import org.springframework.ai.openai.api.OpenAiApi;
```

```
import org.springframework.ai.openai.api.ResponseFormat;
import org.springframework.core.io.Resource;
import org.springframework.stereotype.Service;
import org.springfRamework.util.MimeTypeUtils;
import java.util.Objects;
@Service
public class ScanService {
    private final ChatModel chatModel;
    private final ObjectMapper objectMapper;
    public ScanService(ChatModel chatModel) {
        this.chatModel = chatModel;
        this.objectMapper = new ObjectMapper()
                .registerModule(new JavaTimeModule())
                .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    }
    public FlightInfo scanImage(Resource imageFile) throws JsonProcessingException {
        ResponseFormat responseFormat = ResponseFormat.builder()
                .type(ResponseFormat.Type.JSON_OBJECT).build();
        UserMessage userMessage = new UserMessage("""
                Scan the information in the Flight Ticket.
                Return in json format with keys like following example:
                {
                   "name": "Jane Doe",
                   "flightNumber": "XY1234",
                   "origin": "SFO",
                   "destination": "LHR",
                   "date": "2025-06-01",
                   "time": "14:45:00",
                   "seatNumber": "21C"}
                """,
                new Media(MimeTypeUtils.IMAGE_JPEG, imageFile));
        OpenAiChatOptions openAiChatOptions = OpenAiChatOptions.builder()
                .model(OpenAiApi.ChatModel.GPT_4_O_MINI.getValue())
                .responseFormat(responseFormat)
                .build();
        ChatResponse response = chatModel.call(new Prompt(userMessage, openAiChatO
ptions));
        String jsonResponse = Objects.requireNonNull(
                response.getResult().getOutput().getText()).trim();
        return objectMapper.readValue(jsonResponse, FlightInfo.class);
    }
}
```

The final component to set up is the Controller component. Name it FlightTicketController.java. The functionality of this file is simple enough to fetch the re-

source file and make a call to the service component, as shown in the following code:

File: ./src/main/java/com/example/opencv/controller/FlightTicketController.java

```
package com.example.opencv.controller;
import com.example.opencv.model.FlightInfo;
import com.example.opencv.service.ScanService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.io.IOException;
@RestController
@RequestMapping("/api/flight")
public class FlightTicketController {
    private final ScanService scanService;
    @Autowired
    public FlightTicketController(ScanService scanService) {
        this.scanService = scanService;
    }
    @PostMapping("/scan")
    public ResponseEntity<FlightInfo> scanTicket() throws IOException {
        // 1. Load the image from src/main/resources/flight_ticket.jpg
        Resource imgRes = new ClassPathResource("flight_ticket.jpg");
        // 2. Call the service
        FlightInfo info = scanService.scanImage(imgRes);
        // 3. Return the parsed fields
        return ResponseEntity.ok(info);
    }
}
```

The code is complete at this point. Now is the time to test it with a curl request.

```
curl -X POST    http://localhost:8080/api/flight/scan -H "Accept: application/json"
```

Output:

```
{"name":"JOHNNY BRAVO","flightNumber":"ABC12345","origin":"JAKARTA","destinati
on":"NEW YORK","date":"2025-11-15","time":"06:15:00","seatNumber":"25"}
```

As you can see, the LLM has been successfully able to scan all the information in the image and also present it in JSON format. This showcases the multimodal nature of the use

case where the LLM was able to take in both text and image as input, understand the request in the text, and process the image accordingly.

Generating Images from User Request

Until this point, you have experimented with text to text, image to text. How about text to image? This is where the image-generating capabilities of LLMs shine. There are many use cases where users want to generate images for their applications based on a creative idea they have or modify an existing image and add some enrichment or creativity on top of it.

In this section, we will augment our application to generate a creative image to motivate our customer about the destination place they are visiting and create a collage of all interesting places they could visit there. This seems to be an interesting idea! Let's get on to it.

You will use the existing application setup. You have the destination information fetched by the LLM. The next step is to request the LLM to generate an image with the appropriate prompt as a request to it. To use the image generation feature, you will need to inject the imageModel, which is by default created by Spring AI when your application boots as a bean. You can store a reference to it in the constructor as follows. This will allow you to use the imageModel later in your method calls.

```
public ScanService(ChatModel chatModel, ImageModel imageModel) {
   this.chatModel = chatModel;
   this.imageModel = imageModel;
   this.objectMapper = new ObjectMapper()
         .registerModule(new JavaTimeModule())
         .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
}
```

Let's define a method to achieve the image generation task named generateGreetingImage as follows. It returns the URL of the image generated and takes in as an argument the destination place as string type.

```
private String generateGreetingImage(String destination) {
```

We will then define a prompt to generate this image with the information of the user's destination:

```
String prompt = """
```

```
        As an airline industry, we strive to provide the best
        experience to our customers. After they have booked a
        flight ticket with us, we want to share a collage of
        Some beautiful places present at their destination
        location to inspire and motivate them. Generate an
        image for our customer who is planning to visit: %s
        """.formatted(destination);
```

To generate the image, we have the option to customize some of the attributes like quality, number of images to generate, height and width of the image, etc.

```
OpenAiImageOptions openAiImageOptions = OpenAiImageOptions.builder()
        .withQuality("hd")
        .withN(1)
        .withHeight(1024)
        .withWidth(1024)
        .build();
```

The final step is to instantiate the ImagePrompt with the prompt and image options and call the image model request.

```
ImagePrompt imagePrompt = new ImagePrompt(prompt, openAiImageOptions);
ImageResponse response = imageModel.call(imagePrompt);
return response.getResult().getOutput().getUrl().trim();
```

The end goal is to use this method call from within our scanImage method so that you can pass the destination as an argument to it and update the JSON response you send to your customer with the new attribute. To achieve this, you will also need to update your FlightInfo DTO with this new attribute with a default value of an empty string and respective getter and setter methods. This will enable backward compatibility.

File: ./src/main/java/com/example/opencv/model/FlightInfo.java

```
public class FlightInfo {
    private String name;
    private String flightNumber;
    private String origin;
    private String destination;
    private LocalDate date;
    private LocalTime time;
    private String seatNumber;
    private String greetingImageUrl = "";
```

```
    public String getGreetingImageUrl() {
        return greetingImageUrl;
    }
    public void setGreetingImageUrl(String greetingImageUrl) {
        this.greetingImageUrl = greetingImageUrl;
    }
}
```

Finally, you need to make a call to the generateGreetingImage method, as shown next. We fetch the destination set in the FlightInfo object and then use it to call the generate image method and set the result received through the call to the setter method.

```
FlightInfo flightInfo = objectMapper.readValue(jsonResponse, FlightInfo.class);
String greetingImageUrl = this.generateGreetingImage(flightInfo.getDestination());
flightInfo.setGreetingImageUrl(greetingImageUrl);
```

The following is the full source code of the ScanService.java file for your reference:

File: ./src/main/java/com/example/opencv/service/ScanService.java

```
package com.example.opencv.service;
import com.example.opencv.model.FlightInfo;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.model.ChatResponse;
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.content.Media;
import org.springframework.ai.image.ImageModel;
import org.springframework.ai.image.ImagePrompt;
import org.springframework.ai.image.ImageResponse;
import org.springframework.ai.openai.OpenAiChatOptions;
import org.springframework.ai.openai.OpenAiImageOptions;
import org.springframework.ai.openai.api.OpenAiApi;
import org.springframework.ai.openai.api.ResponseFormat;
import org.springframework.core.io.Resource;
import org.springframework.stereotype.Service;
import org.springframework.util.MimeTypeUtils;
import java.util.Objects;
@Service
public class ScanService {
```

```
    private final ChatModel chatModel;
    private final ImageModel imageModel;
    private final ObjectMapper objectMapper;
    public ScanService(ChatModel chatModel, ImageModel imageModel) {
        this.chatModel = chatModel;
        this.imageModel = imageModel;
        this.objectMapper = new ObjectMapper()
                .registerModule(new JavaTimeModule())
                .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    }
    private String generateGreetingImage(String destination) {
        String prompt = """
                As an airline industry, we strive to provide the best
                experience to our customers. After they have booked a
                flight ticket with us, we want to share a collage of
                some beautiful places present at their destination
                location to inspire and motivate them.  Generate an
                image for our customer who is planning to visit: %s
                """.formatted(destination);
        OpenAiImageOptions openAiImageOptions = OpenAiImageOptions.builder()
                .withQuality("hd")
                .withN(1)
                .withHeight(1024)
                .withWidth(1024)
                .build();
        ImagePrompt imagePrompt = new ImagePrompt(prompt, openAiImageOption
s);
        ImageResponse response = imageModel.call(imagePrompt);
        return response.getResult().getOutput().getUrl().trim();
    }
    public FlightInfo scanImage(Resource imageFile) throws JsonProcessingException {
        ResponseFormat responseFormat = ResponseFormat.builder()
                .type(ResponseFormat.Type.JSON_OBJECT).build();
        UserMessage userMessage = new UserMessage("""
                Scan the information in the Flight Ticket.
                Return in json format with keys like following example:
                {
                   "name": "Jane Doe",
                   "flightNumber": "XY1234",
                   "origin": "SFO",
                   "destination": "LHR",
                   "date": "2025-06-01",
                   "time": "14:45:00",
                   "seatNumber": "21C"}
                """,
                new Media(MimeTypeUtils.IMAGE_JPEG, imageFile));
```

```
        OpenAiChatOptions openAiChatOptions = OpenAiChatOptions.builder()
                .model(OpenAiApi.ChatModel.GPT_4_O_MINI.getValue())
                .responseFormat(responseFormat)
                .build();
        ChatResponse response = chatModel.call(new Prompt(userMessage, openAiChatO
    ptions));
        String jsonResponse = Objects.requireNonNull(
                response.getResult().getOutput().getText()).trim();
        FlightInfo flightInfo = objectMapper.readValue(jsonResponse, FlightInfo.class);
        String greetingImageUrl = this.generateGreetingImage(flightInfo.getDestinatio
    n());
        flightInfo.setGreetingImageUrl(greetingImageUrl);
        return flightInfo;
      }
    }
```

This was a nice plugin component with the existing requirement. Now, let's boot up our application and test it.

```
curl -X POST    http://localhost:8080/api/flight/scan -H "Accept: application/json"
{"name":"JOHNNY BRAVO","flightNumber":"ABC 12345","origin":"JAKARTA","destinat
ion":"NEW YORK","date":"2025-11-15","time":"06:15:00","seatNumber":"25","greetin
gImageUrl":"https://oaidalleapiprodscus.blob.core.windows.net/private/org-f84uKx9
Dr1Qz1LFLH7aNPFwF/user-elDK0kVYtVetb2VDGO2y....."}
```

The response now contains your greeting image URL, and I have shortened it for brevity's sake. You can further store this image in your storage location of choice based on your application requirements. You can click the URL to see the image generated by OpenAI, as shown in Figure 14-7.



Figure 14-7 Open AI generated image for New York as destination, created assistance of GPT-4o Image Generation

Similarly for a customer visiting India, we have the below figure as response as shown in Figure 14-8.



Figure 14-8 Open AI generated image for India as destination, created with assistance of GPT-4o Image Generation

The applications around this could be limitless. I am sure our customer would be very excited to see this image message along with their flight ticket.

Other multimodal formats

This does not have to stop with just one format. We can enrich our inputs with speech from our customer where they can speak on the go and we can pass this to the LLM or the customer is on the go and rather than having them to type, we can process their ask in speech and respond to them back in Speech taking into consideration their security while driving. The use cases are limitless and multimodal LLMs are here to help you.

We will dive deep into audio and video formats but will only touch base on their syntax to give you an idea on them. They can be very similarly, as the above example, be fit into our applications.

In the below example, we want to build an application which detects the emotion behind the user's response and respectively escalate if it's not as to their satisfaction to interact with a live agent. The gpt-4o-audio-preview model which is in preview at this moment can be used to process the audio content and give us the appropriate feedback.

```
Resource userResponseResource = new ClassPathResource("userResponse.mp3");
UserMessage userMessage = new UserMessage("Find out the category of user's respon
se to understand their emotion: POSITIVE, NEGATIVE, NEUTRAL",
    List.of(new Media(MimeTypeUtils.parseMimeType("audio/mp3"), userResponseRe
source)));
ChatResponse response = chatModel.call(
    new Prompt(List.of(userMessage),
        OpenAiChatOptions.builder()
            .model(OpenAiApi.ChatModel.GPT_4_O_AUDIO_PREVIEW)
            .build()
    ));
```

Let's say we want to present some audio to the user based on some logic control and processing of some content passed to the LLM. We can do this using the gpt-4o-audio-preview LLM model as shown in below example:

```
Resource userResponseResource = new ClassPathResource("userResponse.mp3");
User Message userMessage = new UserMessage("Find out the category of user's respo
nse to understand their emotion: POSITIVE, NEGATIVE, NEUTRAL. If NEGATIVE, conve
y the message that we are sorry that their experience has not been positive and that a
n agent would be following up with them soon.",
    List.of(new Media(MimeTypeUtils.parseMimeType("audio/mp3"), userResponseRe
source)));
ChatResponse response = chatModel.call(
    new Prompt(List.of(userMessage),
        OpenAiChatOptions.builder()
            .model(OpenAiApi.ChatModel.GPT_4_O_AUDIO_PREVIEW)
            .outputModalities(List.of("text", "audio"))
            .outputAudio(new AudioParameters(Voice.ALLOY, AudioResponseForma
t.WAV))
            .build()
    ));
String text = response.getResult()
    .getOutput()
    .getContent(); // audio transcript for logging
byte[] waveAudio = response.getResult()
    .getOutput()
    .getMedia()
    .get(0)
```

```
        .getDataAsByteArray(); // audio data for the user
```

This use case is so useful since we do not have to convert the audio to text and text to audio as has been prevalent in the past and also instruct the LLM to work on the audio through text thus enabling multiple input formats to interact to give us the right result. We can also log the transcript for audit purposes to measure and evaluate our LLM for testing purposes.

Conclusion

In this chapter, we journeyed from traditional Java-based image processing—leveraging OpenCV and Tesseract OCR—to the frontier of multimodal AI, where a single large language model can ingest and interpret diverse data formats. By integrating multimodal LLMs into a Spring Boot application, we illustrated how to replace cumbersome pipelines with straightforward API calls that return business-ready JSON and even generate bespoke images tailored to user contexts.

Beyond text and images, we highlighted the extensibility of this approach to audio and video, demonstrating how models in preview can assess emotion from speech or produce synthesized voice responses without manual transcription or text-to-speech conversions. This unified paradigm not only simplifies development but also opens the door to more natural and accessible interfaces, whether for customer support, accessibility applications, or real-time interactive systems.

As multimodal LLMs continue to evolve—increasing in scale, efficiency, and the range of supported modalities—developers will find ever-richer possibilities for creating unified AI experiences. By combining robust Java tooling with these powerful models, you can build applications that see, hear, and reason about the world in concert, bringing us closer to truly holistic, human-centric AI systems.

Now we move on to ML in Java.

Previous chapter

13. Jlama and Friends: Hosting Models the Java Way

Next chapter

15. Does It Even Work? Testing and Evaluating LLM Apps